

RuniqueAppBuilder

Pattern GoF : Builder

Construire une app web pas-à-pas, validée à la fin.

Intention GoF

Séparer la **construction** d'un objet complexe de sa **représentation**, pour construire **pas à pas**.

Le besoin : assembler une app (DB, routes, middlewares, statics, admin, mailer, logs...) où presque tout est optionnel et où l'ordre a des contraintes (middlewares triés par slot, admin mergé avant la stack).

Un constructeur à N paramètres = ingérable → API incrémentale validée à `build()`.

Participants

Rôle GoF	Élément Runique
Builder	<code>RuniqueAppBuilder</code> (état partiel)
étapes	<code>with_database()</code> , <code>routes()</code> , <code>middleware()</code> , <code>with_admin()</code> ... → <code>(mut self) -> Self</code>
Product	<code>RuniqueApp</code> (distinct du builder)
Director	<i>implicite</i> — le <code>main.rs</code>
validation	<code>build()</code> : <code>validate()</code> + <code>all_ready()</code>

Étapes d'accumulation

Chaque `with_*` prend `mut self`, écrit un champ, renvoie `self`. Elles **n'assemblent rien** :

```
new(config)           → RuniqueConfig
  .with_database(db)   → connexion / config DB
  .routes(router)      → Router applicatif
  .middleware(|m| ...) → MiddlewareStaging (sous-builder)
  .with_admin(|a| ...) → AdminStaging      (sous-builder)
  .static_files(|s| ...) → StaticStaging    (sous-builder)
  .with_log(|l| ...)    → RuniqueLog
```

`build()` — la séquence exacte (1/2)

1. Tracing d'abord `log_init + init_subscriber`
2. Validation `validate() + all_ready() → BuildError si KO`
3. DB `core.connect().await`
4. Destructuration `staging (extensions, urls, middleware, statics)`
5. Core (ordre strict) `Tera → password_init → RuniqueEngine`

`build()` est `async` et renvoie `Result<RuniqueApp, BuildError>`.

build() — la séquence exacte (2/2)

```
6.  URLs          add_urls(engine)
7.  Password reset merge router (si configuré)
8.  Admin         prune_orphan_droits + merge router + robots.txt
9.  Middleware    tri par slot PUIS application ← la contrainte clé
10. Statics       attach_static_files (ServeDir + headers)
11. session_db_store
→ RuniqueApp { router, engine } puis run() → axum::serve
```

Particularité 1 — Director implicite

Le GoF canonique : **Director** + **Builder** + **Product**.

Ici, pas de Director explicite → c'est le `main.rs` qui séquence les étapes (forme moderne : « fluent builder »).

Particularité 2 — Builder hiérarchique

`.middleware(|m| ...)`, `.with_admin(|a| ...)` prennent une **closure** `FnOnce(Staging) ->`

Staging :

```
RuniqueAppBuilder
├── CoreStaging      DB, config, URLs
├── MiddlewareStaging  collecte + tri par slot
├── AdminStaging     registry, auth
└── StaticStaging    fichiers, media
```

Un **Builder** composé de sous-Builders spécialisés, eux-mêmes fluents.

Idiome Rust

- Chaque étape **consomme** `self` et **renvoie** `Self` (move) → aucun état de build partagé ; le compilateur interdit de réutiliser un builder à moitié consommé.
- Sous-config par **closure de portée** (`impl FnOnce(X) -> X`) au lieu d'un sous-builder mutable exposé.
- `build()` `async` + `Result<_, BuildError>` → un seul point de validation des invariants.

Tradeoff

Coût	Gain
boilerplate (1 méthode / option)	API incrémentale, config optionnelle sans télescopage de constructeurs
validation différée à <code>build()</code> (erreurs au runtime de construction)	ordre d'assemblage maîtrisé + un seul point de cohérence